07 — The Help System

The in-game `help` command is the player's and builder's window into the world. This chapter documents how it resolves a request, where help content lives, and how to author it — both the curated topic library and the self-documenting verb docstrings that make every command explain itself.

The whole system is one verb — `moo verbs/15/help.py`, defined on `#15` (`BaseRoom`) so it is inherited by every room, OOC and IC alike. There is no separate help engine, no help compiler, and no help database table. Help content is just **object state**: string and dict properties on one object, plus the docstrings already attached to verbs. That is the defining idea of the subsystem — *help is data you set, not code you write.*

Two sources of truth

The `help` verb draws from exactly two places:

Source	What it is	Who authors it
The topic store — `#34` (`help_utils`)	Free-form articles: lore, mechanics, policy, "how to play" prose. Each topic is a property.	Builders / staff (`#34` is owned by the `#100` wizard)
Verb docstrings	The first triple-quoted block of any verb in scope. Documents commands as a side effect of writing them.	Whoever wrote the verb

The split is deliberate. **Commands document themselves** — the same docstring a programmer reads in the source is the help a player reads in the game, so the two can never drift apart. **Everything that isn't a command** — combat rules, the chargen walkthrough, world history, the rules of conduct — lives as editable prose on `#34`, where staff can revise it live without touching a verb.

How a request resolves

`help` runs a fixed resolution ladder and stops at the first rung that matches. Understanding the order is the key to both using and authoring help.

`help` with no argument — the index

Two lists are printed:

1. **Help Topics** — every own property on `#34` except the bookkeeping props `name`, `name_mod_list`, and `description`, and anything whose name begins with an underscore, which is how this object marks its own configuration rather than a topic. Only the object's *own* properties count (`properties_list(include_inherited=False)`), so nothing leaks in from `#34`'s parent.
2. **Command Help** — the visible verbs reachable from where the player is *standing right now*: the verbs on `pobj`, on `pobj.location`, and on every object in the room's contents. A verb is listed only if it has a docstring and the player is allowed to see it (see Visibility).

Because the command list is built from the current scope, **`help` is context-sensitive**: the commands a player sees offered are the commands actually available in that room, on those objects. Walk somewhere else and the list changes.

`help <topic>` — the lookup ladder

For a non-empty argument that does not start with `#`, the verb tries, in order:

Rung	Match	Result
1	Argument equals an own property name on `#34` (case-insensitive)	If the value is a **string**, print it. If it's a **dict**, treat it as a category and print its sorted keys (its sub-topics).
2	Argument equals a **key inside any dict property** on `#34`	Print that sub-topic's text, headed `category > key`.
3	Argument matches a **verb name** in scope (`pobj`, location, contents)	Extract and print the verb's docstring. Respects auth gating.
—	Nothing matched	`There's no help for that.`

So a single word like `combat` can be satisfied by a standalone article, by a sub-topic nested inside a category, or by a command of that name — whichever the ladder reaches first. Author topics with that precedence in mind: a `#34` property name shadows a verb of the same name.

`help #<object>` and `help #<object>.<verb>` — introspection

A leading `#` switches into **object introspection**, gated to `gm3` and above (everyone else gets `There's no help for that.`, so the feature is invisible to players):

- `help #42` — prints `#42`'s `help_text` property, if set. This is a free-form string property you can attach to *any* object to document what it is and how to use it — handy for a complex machine, NPC, or builder fixture.

- `help #42.go` — finds the verb `go` **defined on `#42` itself** (not inherited) and prints its docstring, after checking the viewer's auth against the verb's `auth` level. Use it to read the help for a specific object's override without standing next to it.

Authoring topic articles

Topics are properties on `#34`. Add them in-game with the builder toolkit or from a wizard session.

A standalone article (string property)

`@set` writes a property that already exists; a new topic has to be created first, which is `@adprop`:

```
@adprop #34.rules
@set #34.rules = "Be excellent to each other. No harassment, no cheating..."
```

`help rules` now prints that text. The property name is the topic name, so keep it a single lowercase word or short token — that is exactly what the player types.

For multi-line prose, set it from the verb editor or via `set_property` so you can embed newlines and color codes (e.g. `&<245>` for dim headers, `&n` to reset — see Architecture and the color conventions used elsewhere in the world):

```
set_property(34, 'combat',
    "&<245>Combat&n\n"
    "Attacks are resolved on a roundtime clock. Type 'attack <target>'...\n"
    "See also: help wounds, help parry")
```

A category (dict property)

A property whose value is a **dict** becomes a *category*: an index whose keys are sub-topics. The category name lists its contents; each key is reachable directly.

```
set_property(34, 'magic', {
    'spells':   "Casting consumes mana. Type 'cast <spell> at <target>'...",
    'mana':     "Mana regenerates while resting. Your pool scales with...",
    'schools':  "There are four schools: evocation, abjuration...",
})
```

Now:

- `help magic` → lists `mana, schools, spells` (sorted keys — rung 1, dict branch).
- `help spells` → prints the spells text, headed `magic > spells` (rung 2).

Note the asymmetry: the **category name** (`magic`) appears in the no-argument *Help Topics* index, but the **sub-topic keys** do not — they are discovered by reading the category or by knowing the word. Categories keep the top-level index short while still making every sub-topic directly addressable.

Staff-only topics

A topic can require an auth level. `_topic_auth` on `#34` maps a topic name to the level needed to read it, and a topic not named there is public:

```
@set #34._topic_auth = {"bmatch": 3, "pmatch": 3, "matching": 3}
```

Those three are the shipped example: they document the two matchers a person chooses between *while writing a verb*, which is a decision a player never makes. Below the level, the topic is absent from the *Help Topics* index **and** `help <topic>` answers "There's no help for that" — the same answer as for a topic that does not exist, because "you may not read this" tells someone fishing that there is something there to read. Hiding a topic from the index while still printing it on request is not a gate.

The gate applies to the topic, so a category's sub-topics go with it.

What never becomes a topic

`name`, `name_mod_list`, and `description` on `#34` are filtered out of the topic index, so you can name and describe the help object normally without those showing up as bogus topics.

Authoring command help

A command documents itself through its **docstring** — the first `"""..."""` (or `'''...'''`) block in the verb's source. The help verb extracts the text between the opening and closing triple-quotes, strips it, and prints it verbatim. No annotation, registration, or separate help entry is required: write the docstring and the command is documented.

The house style — visible in nearly every shipped verb — is a one-line summary, a `Usage:` line, and optional examples:

```
"""
Jump across a gap or obstacle.

Usage: jump <exit>
"""
```

`help jump` (while standing where `jump` is available) prints that block. Because the docstring is the same text the next programmer reads, keeping help accurate is the same act as keeping the source readable.

Visibility: hidden and gated verbs

The help verb honors two verb attributes so help never advertises a command the player can't use or shouldn't see:

- **`hidden`** — a verb flagged hidden (e.g. by `@hideverb`) is omitted from the no-argument *Command Help* index. It can still be looked up by name if you know it, but it isn't broadcast.
- **`auth`** — if a verb declares a minimum auth level, players below that level see neither the verb in the index nor its docstring on direct lookup; the lookup falls through as though the verb weren't there. Staff-only commands therefore stay invisible to players in the help system automatically. (`auth_level(pobj)` supplies the viewer's level; the permission ladder is in Operations.)

This is why command help is trustworthy: the list a player sees is, by construction, the list of commands available *and* permitted to them, in their current location.

Worked example: adding a "factions" topic

A builder wants `help factions` to exist as a category with three entries.

```
# from a wizard eval / verb editor
set_property(34, 'factions', {
    'guild':   "The Tinkers' Guild controls trade in the eastern wards...",
    'crown':   "The Crown's agents enforce the King's law...",
    'free':    "The Free Companies answer to coin and little else...",
})
```

Verification, the way a player would experience it:

```
help                -> "factions" now appears under Help Topics
help factions       -> lists: crown, free, guild
help crown          -> prints the crown text, headed "factions > crown"
```

No restart and no verb edit — the topic is live the moment the property is set, because help content is object state read on every invocation.

Quick reference

Command	Who	Effect
`help`	all	Index: topic names on `#34` + visible commands in scope
`help <topic>`	all	Article (string), category index (dict), sub-topic, or command docstring — first match wins
`help #<obj>`	gm3+	Print the object's `help_text` property
`help #<obj>.<verb>`	gm3+	Print the docstring of that verb as defined on that object

To author…	Do this
A standalone article	Set a **string** property on `#34`: `@set #34.<topic> = "..."`
A category of sub-topics	Set a **dict** property on `#34`: `set_property(34, '<cat>', {...})`
Command help	Write a triple-quoted **docstring** as the first thing in the verb
Help for an object	Set a `help_text` string property on that object
Hide a command from the index	Flag the verb `hidden` (e.g. `@hideverb`)
Restrict help to staff	Give the verb an `auth` level

Design notes and gotchas

- **One object holds all articles.** `#34` (`help_utils`, owned by `#100`) is the entire topic library. Back it up with the rest of the database; there is no separate help store to manage.
- **Precedence matters.** A `#34` property name shadows a verb of the same name (rung 1 before rung 3). If you add a `look` property to `#34`, `help look` stops reaching the `look` command. Name topics so they don't collide with commands unless you intend to override them.
- **Sub-topic keys are not indexed.** They are addressable (`help <key>`) but do not appear in the no-argument list — only their category does. Mention important sub-topics in the category's own prose, or in a "See also:" line, so players can find them.
- **Command help is local.** The no-argument index reflects the current room and the objects in it. A command defined on an object in another room won't appear until the player is in scope of it. This is a feature — it surfaces what's usable here — but don't expect a global command list from `help` alone.

- **Help is read live.** Every `help` invocation re-reads `#34` and the verbs in scope, so edits to topics or docstrings take effect immediately, with no reload.
- **The login banner points here.** New connections are greeted with *"Type 'help' for help, or 'quit' to disconnect."* (`moo/globals.py`, `moo/server.py`), making this verb the documented front door to the world.